



SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Techniques for Using Neural Architecture Search in Federated Learning

Max Coetzee





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

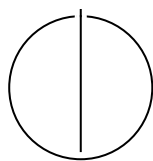
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Techniques for Using Neural Architecture
Search in Federated Learning**

**Techniken um Neurale Architektursuche für
Föderatives Lernen zu nutzen**

Author:	Max Coetzee
Examiner:	Supervisor
Supervisor:	Nick Henze, Niclas Kannengießer
Submission Date:	30.07.2026



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 30.07.2026

Max Coetzee

Acknowledgments

Abstract

Both *Neural Architecture Search* (NAS) and *Federated Learning* (FL) have made significant progress independently in the past decade. To benefit from the advantages of NAS methods in FL, researchers have started combining them by using NAS in FL [fednas_2021] [fedoras_2022] [finch_2024].

[TODO: overhaul]

Applying techniques from Neural Architecture Search (NAS) to Federated Learning (FL) has been fruitful (remove) in recent years. The combination was identified as a promising research (remove) direction by [fl_advances_and_open_problems_2021]. It has yielded methods for finding architectures that deal with the challenges imposed by the FL setting.

Research into NAS has grown rapidly [nas_1000_papers_2023] since it was popularized by [nas_with_rl_2017]; consequently, literature on its application to FL has grown. The last survey on NAS applied to FL compared approaches of four papers [fl_to_nas_survey_2021]. Since then, we have identified approximately 50 new papers. This motivates a new systematic survey of the landscape to identify progress and gaps in the literature.

In this thesis, we propose a map of the literature landscape based on the FL challenges they address. We achieve this by systematically evaluating the literature and identifying which challenge it solves.

We refer to the FL challenges described in [nas_with_rl_2017], i.e., non-IID data, limited communication, client heterogeneity, privacy of client data, and break them down into smaller subchallenges — each subchallenge being associated with a pattern in the literature. We include personalized FL [personalized_fl_2023] as an additional subchallenge that was not originally posited, but has since drawn the community’s attention.

We then analyze how the subchallenges are addressed and focus on the contribution of the used NAS method towards overcoming the subchallenge. For each subchallenge, we keep track of the NAS types used (following [nas_1000_papers_2023], [systematic_nas_survey_2024]) and assess whether the underexplored methods are candidates for future research.

Neural Architecture Search

Contents

1 Introduction

Deep learning has seen widespread adoption over the last decade and is being applied to an increasingly diverse set of tasks. Applying deep learning traditionally involves a team of deep learning and domain experts who tailor a neural network architecture to a specific task through a lengthy process of trial and error. To automate this process, researchers invented *Neural Architecture Search* (NAS) [nas_survey_2019] methods. NAS methods employ diverse strategies to automatically search for a neural network architecture for a given deep learning task within an architecture search space.

Independently, but in parallel to NAS, researchers developed a machine learning approach called *Federated Learning* (FL) [fl_semi_2017], which enables training an ML model on data distributed across a set of clients without sharing their data. The model is distributed from the server to the clients, which train the model on their local data. The clients then send the model weight updates to the server, which aggregates them and redistributes them to the clients. This process is repeated for several rounds. FL enhances the privacy of client data and enables training even when clients' data is too large to transfer effectively to a central training site or cannot be shared due to regulatory or privacy constraints.

There has been significant interest in using NAS in FL over the last couple of years, spawning *NAS-in-FL methods* [nas_in_fl_2026], but using NAS methods in FL is not straightforward. Most NAS methods were developed for a centralised setting and can not be applied directly in FL. For example, in a centralised setting, NAS methods typically run on worker nodes connected via low-latency, high-bandwidth links. However, in FL, a NAS method may run on clients connected via an unreliable, high-latency, low-bandwidth network. This creates a *challenge* when transferring the weights of large candidate models between clients and the server to coordinate an architecture search. Specialized solutions are needed to overcome such challenges for NAS-in-FL methods, such as only transferring encodings of candidate architectures to clients [aging_fednas_2025].

The NAS-in-FL literature proposes many solutions to address these challenges, but their mechanisms are often obscured because they are bundled as a part of a NAS-in-FL method. Earlier reviews classify NAS-in-FL methods by properties such as when search occurs and which objectives it optimizes [fl_to_nas_survey_2021], summarize selected methods within broader reviews [nas_hpo_fl_survey_2023], or discuss NAS

as part of multi-objective FL [**multi-objective_methods_in_fl_2025**]. More recent work supports a more fine-grained comparison through a generic NAS-in-FL pipeline, design elements, and method archetypes [**nas_in_fl_2026**]. These contributions organize whole NAS-in-FL methods and their configurations, but recurring solution mechanism and the challenges they address remain scattered across the literature.

Examining the mechanisms of solutions to challenges individually would make it possible to assess their potential for reuse in new NAS-in-FL methods, but solution mechanisms exist at different levels of abstraction and use different terminology throughout the literature. A synthesis of recurring challenges in the NAS-in-FL literature and their solution mechanisms is needed as well as an understanding of their applicability across NAS-in-FL methods. This leads us to our research question:

Can solution mechanisms identified in the NAS-in-FL literature be re-used?

To answer our research question, we first extract a coherent set of challenges and their related solutions and arrange them around central challenge themes.

We propose grouping challenges and solution around central themes (or should it be around which parts of the NAS-in-FL scenario activates it?) recurring combinations of FL system characteristics and NAS method types which we call NAS-in-FL scenarios. NAS-in-FL scenario is part of my contribution.

To tackle our research question, we conduct an extensive systematic literature review. We collect an initial set of relevant literature with a Scopus [**scopus**] search and extend it with a snowball search. We first use open coding to identify, from the literature, assumption discrepancies and resulting challenges, when using NAS in FL. Next, we extract FL system characteristics and use axial coding to code the influence of characteristics on the relevance of challenges. Then, we extract techniques through open coding and iteratively refine them to synthesise a coherent set of mutually exclusive and collectively exhaustive techniques. Finally, we use axial coding to map techniques onto challenges they overcome.

Our review aims to synthesise techniques for using NAS in FL from the literature to make finding suitable techniques easy in practice. We make the following three contributions. First, by providing a consolidated overview of the challenges of using NAS in FL, practitioners can grasp which issues they can expect to encounter at a glance. Second, by describing how FL system characteristics influence the relevance of challenges, we enable practitioners to focus on the challenges relevant to them. Third, by extracting techniques for using NAS in FL and highlighting the challenges they address, we enable practitioners to compare existing techniques for using NAS in FL for their specific set of challenges.

In Chapter ??, we cover the background required for this thesis and related work. In Chapter 3, we describe the method with which we perform our literature review and give an overview of the included literature. In Chapter ??, we highlight the

challenges with FedNAS and how they arise from different FL settings. In Chapter ??, we describe the adaptation techniques we synthesised and how they overcome challenges. In Chapter 7, we conduct a discussion about our work. Chapter 8 contains our conclusion.

2 Background

2.0.1 Neural Architecture Search

Neural architecture search (NAS) automates the design of neural network architectures. A NAS method consists of a search space, a search strategy and a performance-estimation strategy [nas_survey_2019]. The search space determines the potential candidate architectures, and the search strategy decides which candidates to evaluate next based on the performance-estimation strategy.

NAS Method Type

We define a NAS method type as a triple comprising a search-strategy type, a search-space type and a performance-estimation type.

We group *search strategies* into black-box optimization, supernet-based methods and other strategies. Black-box optimization includes reinforcement learning, evolutionary and genetic algorithms, Bayesian optimization and Monte Carlo tree search. Supernet-based methods are differentiable when gradients optimize continuous architecture parameters and non-differentiable when discrete subnetworks are selected without such parameters [nas_1000_papers_2023].

Search spaces can be cell-based, hierarchical, chain-structured or other. Cell-based spaces search reusable computational blocks, hierarchical spaces compose structures across multiple levels and chain-structured spaces vary a sequential series of layers.

Performance estimation is classified as observed validation performance after brief training, surrogate-predicted performance or another estimator. Brief training is the predominant approach in the reviewed NAS-in-FL literature.

2.0.2 Federated Learning

Federated learning (FL) enables clients to train a model collaboratively while keeping their raw data local. A server distributes a global model to selected clients, which train it locally and return updates for aggregation [fl_seminal_2017]. Cross-device FL generally involves many resource-constrained and intermittently available devices, while cross-silo FL involves fewer, more reliable organizations. Data may also be

partitioned horizontally by samples or vertically by features. These system characteristics shape the resources, coordination and trust available to the federated process [fl_advances_and_open_problems_2021].

2.0.3 NAS in FL

NAS in FL performs an architecture search through the federated process without centralizing client data. Candidate evaluation is performed on the client, while candidate generation, evaluation, search-state updates and selection may occur at the server, the clients or both [nas_in_fl_2026]. Depending on the search strategy, clients exchange model weights and search state for separate candidates or a shared supernet. This adds resource and coordination demands to fixed-model FL. The applicability of a challenge or solution therefore depends on both the FL system characteristics and the NAS design.

NAS-in-FL Scenario

We introduce the term *NAS-in-FL scenario* for a tuple comprising a NAS method type and a set of FL system characteristics based on [fl_advances_and_open_problems_2021, fl_in_practice_reflections_2024]:

- *Data heterogeneity*: homogeneous or heterogeneous client distributions
- *Data balance*: balanced or imbalanced client data quantities
- *Client computation resources*: homogeneous or heterogeneous capacities
- *Network environment*: homogeneous or heterogeneous communication conditions
- *Federation environment*: stable or dynamic client availability
- *Deployment environment*: uniform or heterogeneous deployment requirements
- *Trust and privacy requirements*: data locality only or additional confidentiality or integrity requirements

The term provides a common unit for specifying where a challenge arises and where a solution is applicable. Broad labels such as cross-device and cross-silo describe representative combinations of FL system characteristics rather than exhaustive system types [fl_advances_and_open_problems_2021]. NAS-in-FL methods also differ in how they distribute candidate generation, evaluation, search-state updates and selection between the server and clients [nas_in_fl_2026]. Describing a scenario through both the

NAS method type and individual FL system characteristics therefore avoids assuming that a challenge or solution applies to every method in the same broad FL setting. It also allows an applicability claim to constrain only the characteristics on which its mechanism depends while leaving the others unrestricted.

3 Method

We conduct a literature review following [webster_watson_literature_review_2002] and analyse the literature on the basis of thematic analysis to extract challenges and solutions for using NAS in FL.

3.1 Literature Selection

We start our literature selection with a search across titles, abstracts and keywords of English-language publications indexed by Scopus using the query "federated learning" AND "neural architecture search" [scopus]. A publication is eligible when it describes a method in which NAS is performed in FL. We excluded publications that address only NAS or FL independently and publications in which centrally completed NAS merely preceeds FL without NAS. Literature review are excluded from the analysis, but are discussed in related work.

For inclusion in the thematic analysis, a publication had to provide evidence from which at least one relevant challenge or solution could be coded. A challenge met this threshold when the publication connected a condition of FL to a consequence for architecture search. A solution met it when the described procedure contained a deliberate intervention whose effect on the NAS-in-FL process could be identified. General claims that an approach addressed concerns such as heterogeneity, efficiency or privacy without explaining the relevant consequence or intervention did not meet this threshold.

We extended the initial set through backward and forward citation searching of the eligible publications, as recommended by [webster_watson_literature_review_2002]. Newly identified publications were screened using the same criteria, and citation searching ended when an iteration yielded no additional eligible publications.

TODO: include prisma chart to reduce prose significantly (applicable with webster and watson?)

3.2 Literature Analysis

3.2.1 Extracting Challenges and Solutions

- analyzed literature using inductive thematic analysis as described in [using_thematic_analysis_2006]
- for familiarisation we read papers - for initial codes we used the concept matrix proposed by [webster_watson_literature_review_2002] and create a row for each NAS-in-FL method where we code all passages that a) indicate FL system characteristics, b) indicate the NAS method type, c) show challenges that are due to using NAS in FL and d) solutions for these challenges

TODO: read what we did in the split and merge rounds of markdown files to flesh this method description out and map it onto thematic analysis - then we iteratively refined challenges and solutions until internal homogeneity and external heterogeneity as described in ...: - we do multiple iterations of merging, splitting up and clarifying boundaries between challenges and solutions as well discarding challenges/solution which make weak claims or are not relevant to our research problem as recommended in ... - remove challenges which look like a challenge related to NAS-in-FL at first, but are actually general challenges - challenges are int. hom. and ext. het. when they make just one analytical claim relevant to our narrative - solutions are int. hom. and ext. het. when they consist of just a single mechanism that ... - we then attach solutions to challenges based on (how do we decide if a solution belongs to a challenge? TODO: come up with approach)

- TODO: how did we determine that a challenge is addressed by a solution? similar to smart contracts paper?

- next we derive relevant FL system characteristics and NAS method types - TODO: How did we decide which characteristics to include and how to classify NAS method types?

4 Related Work

Existing reviews have made important contributions to organizing the growing body of research on NAS in FL. Their classifications explain broad procedural differences between approaches and, more recently, the design decisions from which these differences arise. However, their primary unit of analysis is the complete NAS-in-FL method or its design configuration. The challenges that motivate individual design decisions and the solution mechanisms shared across methods consequently remain distributed throughout the primary literature.

Earlier reviews organize NAS-in-FL research around broad procedural and optimization categories. One review distinguishes online from offline implementations and single-objective from multi-objective searches, thereby showing when architecture search occurs and which objectives guide it [fl_to_nas_survey_2021]. A broader review compares NAS and hyperparameter optimization in centralized and federated environments and summarizes how individual approaches modify conventional NAS components for FL [nas_hpo_fl_survey_2023]. Research on multi-objective methods in FL offers an adjacent perspective by considering architecture search where it contributes to the joint optimization of predictive performance and system objectives [multi_objective_methods_in_fl_2025]. These classifications provide an overview of the available approaches, but they do not consolidate the recurring challenges and solutions embedded within them.

More recent work provides a finer-grained account of NAS-in-FL design. A unified design concept decomposes the NAS-in-FL pipeline into design dimensions, identifies recurring method archetypes and relates their configurations to method traits and suitable FL systems [nas_in_fl_2026]. This representation supplies a common vocabulary for comparing approaches that use different terminology and distribute search activities differently between clients and the server. Its analytical focus remains the configuration of complete approaches, leaving the relationships between particular challenges and reusable solution mechanisms implicit within those configurations.

Reviews of NAS and FL separately provide concepts needed to understand these relationships. NAS research distinguishes the search space, search strategy and performance-estimation strategy [nas_survey_2019], while FL taxonomies organize concerns such as heterogeneity, efficiency, security and aggregation [fl_taxonomy_2024]. Each describes one side of the combined setting. In NAS-in-FL, their interaction

can change the problem. For example, the cost of candidate evaluation becomes a distributed client workload, while differences between searched architectures can invalidate the shared representation assumed by conventional aggregation [nas_in_fl_2026, rafl_2024]. These adjacent reviews therefore establish relevant context without providing an account of the challenge and solution space created by their combination.

This thesis complements the existing classifications by using recurring challenges and solutions as its units of synthesis. It consolidates descriptions that occur under different terminology, distinguishes solutions by their mechanisms and maps the relationships between them. The analysis also retains the FL system characteristics and NAS designs that delimit where a challenge arises and where a solution is applicable. This concept-centric perspective explains why particular changes to conventional NAS are needed and makes visible how the same solution can recur across otherwise different NAS-in-FL methods.

5 Challenges and Solutions for Using NAS in FL

Our thematic analysis revealed ten key challenges acting as sub-themes of five themes relating to FL system characteristics: *client resource constraints*, *client heterogeneity*, *decentralized search state*, *participation dynamics*, and *security*.

A challenge may have several solutions, and one solution may address several challenges. For clarity, each solution appears under the challenge it primarily addresses. Links to other challenges are identified as cross-cutting.

We generally qualify challenges and solutions with the NAS method types of the publications they originate from.

5.1 Client Resource Constraints

Training many candidate architectures from scratch makes NAS computationally demanding [nas_1000_papers_2023]. FL clients often have much less compute and memory available than their centralized counterparts. The challenges below deal with this difficulty.

Challenge 1: Client Capacity Limits the Search Work Assigned Per Round

NAS-in-FL becomes infeasible when an assigned search workload exceeds a client’s compute or memory. Hardware differences compound this problem because the same workload may fit one client but not another [fedoras_2022, fednas_over_hetero_mobile_devices_2022, dfnas_2020, peaches_2024, supernet_trade-off_fednas_2025]. Differentiable supernet-based strategies are particularly demanding because clients update several candidate operations within a shared supernet [fednas_2021, dp-fnas_2020, feathers_2023, fed_auto_mri_2023]. Other strategies usually assign one candidate at a time, but even that candidate can exceed a low-end client’s capacity. This challenge concerns whether clients can execute their assigned task. Challenge 9 concerns how long capable clients take to complete it.

Solution 1a: Use resource-efficient search components. Practitioners can restrict the search space to inexpensive components, such as depthwise-separable convolutions and

mobile-oriented CNN blocks [fed_auto_mri_2023, intrusion_detection_fednas_2024]. This makes every candidate cheaper to train, but may exclude costly components that improve prediction accuracy. Solution 10a instead tests candidate modules on server data.

Solution 1b: Use capacity-compatible subnets or paths. Rather than make every client train the complete supernet, the search can assign a subnet or active path that fits the client’s limits. Its depth, width, channel count or sampling budget can reflect available compute and memory [fedoras_2022, divide-and-conquer_fednas_2023, peaches_2024, fgfl_2024, supernet_trade-off_fednas_2025, perfedrlnas_2024]. Single-path local search provides a related way to reduce memory use [dfnas_2020, efnas_2024, load_monitoring_fednas_2025]. As a result, some operations or widths may receive fewer updates, as discussed in Challenge 6.

Solution 1c: Search trainable adaptation modules over a frozen backbone. When a suitable pretrained model is available, clients can keep its backbone fixed and search for a small combination of lightweight adaptation modules. Only these modules and their architecture parameters require local training and exchange, reducing computation and update size [fednaspet_2023]. The benefit depends on how well the fixed backbone transfers to the clients’ tasks.

Cross-cutting solution 7d. Architecture-agnostic distillation lets small client models teach a server-held supernet without requiring clients to run it [hapfnas_2025].

Challenge 2: Architecture Search Adds Computational Cost

Even when clients can handle each round, the complete architecture search can take too long. Black-box strategies may train several candidates in separate federated evaluations [decnas_2022, rt_fed_evo_nas_2020]. Weight-sharing strategies instead repeatedly update a large supernet or sampled subnets [fednas_2021, superfednas_2024]. Split-model execution can also leave clients idle while other segments run [nasfly_2024]. Some strategies discard the search-trained weights and retrain the selected architecture from a new initialization, adding more local epochs and aggregation rounds [medical_image_segmentation_privacy_preserving_fednas_for_edge_2025]. This challenge arises from repeated candidate evaluation, idle resources or search work that produces no reusable weights. Challenge 3 addresses communication volume.

Solution 2a: Use multi-fidelity evaluation for candidate architectures. Black-box strategies can first evaluate each candidate with only a few communication rounds. Intermediate validation performance determines which candidates stop early and which receive more rounds. This multi-fidelity approach is used across several black-box search strategies and follows established AutoML methods [automl_book_hpo_2019].

For example, [decnas_2022, fednas_over_hetero_mobile_devices_2022] report that

the best candidate usually becomes distinguishable within the first few rounds. Their procedures drop weaker candidates and increase the budget in later search iterations. Some evolutionary strategies likewise use a small fixed budget or early stopping [multi_objective_evo_fl_2020, gpt4-boosted_pso_fednas_2024].

Solution 2b: Prune a supernet progressively. Supernet-based strategies can periodically remove weak operations, leaving fewer alternatives to store and train in later rounds [dfnas_2020, peaches_2024]. Unlike changing the evaluation budget, pruning reduces the search space itself.

Whether the final model needs separate training depends on how its weights are retained, as addressed by Solution 2e and Solution 2g. Pruning saves more work as search progresses, but early rounds still train the complete supernet and premature pruning can remove useful operations.

Solution 2c: Predict candidate performance. Performance prediction reduces costly configuration evaluations in AutoML [automl_book_hpo_2019]. NAS-in-FL strategies can evaluate a small set of architectures and train a predictor to estimate the quality of the rest [hapfnas_2025, energy_fednas_2025].

Because private, non-IID data can produce different rankings at each client, a separate predictor can be trained for each client [hapfnas_2025]. Predictors can also be learned collaboratively without transmitting locally evaluated architectures [federated_bayesian_optimization_na]. This reduces client evaluations, but works only if the evaluated architectures represent the relevant search-space regions.

Solution 2d: Evaluate independent candidates or subnets in parallel. Black-box strategies can train and evaluate different candidates concurrently on separate client groups [decnas_2022, rt_fed_evo_nas_2020, energy_fednas_2025].

Supernet-based strategies can sample several subnets, assign each to a client group and merge their updates into the shared supernet [finch_2024]. This can reduce search time when enough clients are available, but different group data can bias candidate comparisons [decnas_2022]. Challenge 6 addresses this problem.

Solution 2e: Reuse search-trained weights for candidate evaluation and deployment. In weight-sharing strategies, candidates can inherit the operation weights associated with their architecture [fl-agtnns_2023, optimize_fednas_edge_2021]. This avoids training every candidate from scratch and can initialize the selected subnet before limited fine-tuning [medical_image_segmentation_fednas_2024]. Here, architecture selection remains separate from supernet training. Solution 2g instead produces the architecture and deployable weights together. Shared weights may still rank candidates poorly or provide a weak final initialization.

Solution 2f: Use split-training idle periods for local search. Temporary input and output components can make an assigned model segment executable on its own. During idle periods, a client can then sample and train alternative branches without

waiting for the other model segments [nasfly_2024]. The assigned segment can also provide the feature-based teaching signal described in Solution 7d. This converts waiting time into search work.

Solution 2g: Jointly derive an architecture and its deployable weights. One-stage supernet strategies can train model weights while narrowing the architecture choices until the search produces a discrete model with trained weights [dfnas_2020, fed_meta_nas_2025]. Reinforcement-learning strategies over a shared supernet can similarly use later rounds to fine-tune the final model as sampling concentrates on one path [perfedrlnas_2024]. Unlike Solution 2b, this makes final model production part of search. The selected path may remain undertrained if competing paths received most updates.

Challenge 3: Architecture Search Increases Communication Costs

FL with a predefined architecture exchanges one model’s parameters. NAS-in-FL may send larger updates, synchronize more often and exchange many candidate models. A supernet stores weights for alternative operations and can be much larger than the selected architecture. Differentiable strategies may also send architecture parameters throughout search [fedoras_2022, hapfnas_2025, fednas_2021, fedregnas_2025].

Solution 3a: Use compact codes to identify known candidates. When clients already hold a compatible master model or trained supernet, the server can send a compact selector or genotype instead of the candidate’s parameters. Clients reconstruct the subnet locally and return its evaluation or updated weights [rt_fed_evo_nas_2020, medical_image_segmentation_fednas_2024, intrusion_detection_fednas_2024]. Unlike Solution 7b, this requires a shared search space and common retained weights.

Solution 3b: Synchronize architecture parameters less frequently. Clients can update and upload architecture parameters less often than model weights. One approach transmits them every H_α rounds and stops once the architecture is fixed, while weight training continues [fedregnas_2025].

Solution 3c: Aggregate search updates hierarchically. A nearby aggregator can combine updates from several clients before forwarding one result to the remote server. This shifts most aggregation to local links and reduces the search traffic transferred over the wider network [finch_2024].

The selected architecture also determines later communication costs. A model chosen only for predictive performance or local computation may have many parameters that FL must transmit in every training round. It can therefore meet compute and memory limits while exceeding the transfer budget [multi_objective_evo_fl_2020]. This challenge covers communication during search and during later training of the selected model.

Cross-cutting solution 4b. This solution can include model size or expected transmitted

bytes in candidate scores, allowing the search to consider later communication costs before selecting an architecture [multi_objective_evo_fl_2020, fedregnas_2025].

5.2 Client Heterogeneity

Clients can differ substantially in compute, network capacity and data distributions. These differences affect which architectures are suitable and how candidate updates should be interpreted.

Challenge 4: No Single Architecture Meets All Deployment Requirements

A NAS-in-FL deployment may include devices with different inference-time compute, memory and latency limits. An architecture sized for the weakest client can underuse stronger hardware, while one sized for the strongest client may not run elsewhere [fedoras_2022]. The search must therefore produce architectures for several targets. Running an independent federated search for each target repeats computation and communication, so costs grow with the number of device groups or deployment budgets [superfednas_2024].

Solution 4a: Train one supernet for different deployment constraints. A weight-sharing supernet can train subnets with different depths, widths or exits in one federated process. Clients with similar limits can receive one architecture per device group [fedoras_2022, clustered_direct_fednas_2022]. Finer-grained selection can instead choose a subnet for each client’s hardware limits and validation data [superfednas_2024, hapfnas_2025, enasfl_2024].

This solution selects the deployed model, while Solution 1b adjusts the workload a client executes during search. In [superfednas_2024], local predictor can select subnets without further federated training, reducing training cost by a factor of 11 for 20 targets in one evaluation, but training many subnets together can cause interference and leave rarely sampled parameters stale.

Solution 4b: Include resource costs in the architecture search objective. Supernet-based strategies can score predictive performance together with measured deployment costs. Latency or operation-count limits can tailor the result to each client group’s hardware [clustered_direct_fednas_2022, load_monitoring_fednas_2025, fgfl_2024, privacy-preserving_nas_2025]. The score can also penalize model size or expected bytes transmitted during later federated training [multi_objective_evo_fl_2020, fedregnas_2025]. Unlike Solution 1a and Solution 10a, this changes how candidates are scored without removing them. The search can therefore balance prediction quality, inference limits and later communication before selecting an architecture.

Challenge 5: Data Heterogeneity Creates Conflicting Architecture Updates

Architecture updates depend on each client’s data. Differences in class balance and features can make clients favor different operations or rank the same candidates differently [collaborative_nas_for_pfl_2025, superfednas_2024]. Clients solving related tasks may also require different outputs [fedpnas_2021]. Architecture preferences can also vary over time as client data regimes change [fedregnas_2025].

Solution 5a: Cluster clients by learned architecture preference. Clients with similar locally learned architecture parameters can form a group and train one architecture together [electricity_load_forecasting_fl_darts_2023]. Unlike grouping by latency or data balance, this mechanism groups clients by their learned architecture preferences.

Solution 5b: Optimize the worst-performing subnets across client groups. Instead of minimizing only average loss, the search can focus on the subnet with the highest loss in each client group. Sampling the smallest, largest and random subnets can approximate this objective [superfednas_2024]. Giving more attention to poorly performing client-subnet pairs prevents easier pairs from dominating shared-weight training.

Solution 5c: Split the architecture into shared and task-specific components. When clients solve related tasks, the search space can separate a shared feature extractor from task-specific outputs. Clients collaborate on the shared structure while selecting and fine-tuning local components for their task [fedpnas_2021].

Cross-cutting solution 6b. Grouping clients to improve data coverage can make each candidate’s evaluation better reflect the target population [finch_2024].

Challenge 6: Capability-Based Assignment Biases Candidate Evaluation

Assigning only subnets that fit each client’s hardware allows more clients to contribute, but also determines which parts of the supernet they train. Operations and widths found only in large subnets may receive updates from few capable clients, while unselected parameters receive none [fedoras_2022, peaches_2024, supernet_trade-off_fednas_2025]. Treating all parameters as equally trained can then distort the shared search state.

With non-IID data, assignment also ties each model size to the data of clients that can run it. Small candidates may train mainly on low-capability clients and large candidates on well-connected clients [fedoras_2022]. Candidate scores may then reflect the assignment pattern rather than architecture quality, even when every selected client returns an update [hapfnas_2025]. Unequal training and biased data coverage are separate effects of the same assignment process.

Solution 6a: Aggregate each parameter only from clients that trained it. The server can aggregate an operation only from clients that updated it and weight their updates by the number of examples processed by that operation [fedoras_2022, fgfl_2024,

supernet_trade-off_fednas_2025]. The resulting aggregation weights differ by parameter and reflect how much training each received. This avoids treating missing parameter updates as ordinary client contributions.

Solution 6b: Form groups based on resources and representative data. Client groups can meet a time or bandwidth limit while jointly representing the target class distribution. Methods either rebalance bandwidth-based groups by class distribution or minimize each group’s difference from the population under a completion-time limit [**network_aware_fed_nas_2025, finch_2024**]. Forming a group for each candidate makes comparisons less dependent on which clients can run it. The same approach can keep recurring client availability from biasing round-level participation.

Cross-cutting solution 7d. Architecture-agnostic distillation lets candidates learn from client predictions or features even when those clients cannot run the candidates [**hapfnas_2025, collaborative_nas_for_pfl_2025**].

5.3 Decentralized Search State

NAS-in-FL distributes architecture parameters and model weights across clients and rounds. The resulting challenges concern whether these updates can be combined meaningfully and whether candidate evaluations remain comparable as the FL setup and architecture change.

Challenge 7: Distributed Search States Cannot Always Be Averaged

Differentiable NAS trains model weights and architecture parameters together in a shared supernet. The architecture parameters control which operations contribute, and their updates depend on the current operation weights. Aggregating only model weights loses local architecture decisions. Combining architecture parameters with unrelated weights separates those decisions from the state that produced them [**fednas_2021, dfnas_2020, feathers_2023**]. Direct aggregation therefore assumes that every client uses the same supernet and that its variables have the same meaning.

Independent local searches can return graphs with different connections, operations or blocks. Because their variables refer to different structures, ordinary averaging can lose architecture information [**multi-granular_fednas_2023, efnas_2024**]. Aggregating only shared components also discards knowledge from components unique to a local architecture [**f2nas_2024**].

Solution 7a: Aggregate matching architecture parameters and model weights together. When clients use the same supernet, they can return both architecture parameters and model weights after local search. The server aggregates and redistributes them together [**fednas_2021, dfnas_2020, feathers_2023**]. This maintains one shared search

state across rounds, but requires each variable to have the same meaning and shape at every client.

Solution 7b: Aggregate architecture encodings separately from model parameters. Clients can upload compact graph encodings without model weights. The server counts how often connections and operations occur and uses these frequencies to sample a global graph [multi-granular_fednas_2023]. A related strategy assembles frequently recurring subgraphs directly [efnas_2024]. Both preserve architecture information without matching all local model parameters. Compact encodings may contain fewer values than continuous gradients, but provide no formal privacy guarantee.

Solution 7c: Combine matching blocks based on distribution similarity. When different local architectures contain compatible blocks, the server can give more weight to blocks whose parameter distributions resemble that of the server block. Similarity is measured with Jensen–Shannon divergence [f2nas_2024]. This transfers parameters between matching blocks without assuming that the complete architectures match.

Solution 7d: Transfer knowledge between architectures through distillation. Client predictions or feature maps on shared inputs can teach a server supernet, sampled subnets or different blocks. A target architecture can therefore learn from clients that cannot run it or whose parameters do not match [f2nas_2024, hapfnas_2025, collaborative_nas_for_pfl_2025]. Compressed predictions can likewise support collaboration between entirely different architectures [hetefed_2023]. This avoids matching parameters directly, but requires shared inputs and transfers model behavior rather than complete parameters.

Challenge 8: Architecture Evaluation Depends on the FL Configuration

Because raw client data remain local, candidates cannot usually be trained and validated on the same pooled data [fl_advances_and_open_problems_2021]. Their scores instead depend on the FL setup, including the number of local epochs and the fraction of participating clients. An architecture selected under one setup may rank differently under another [feathers_2023, aging_fednas_2025, breastcancer_fednas_2024]. Changing these settings after search can therefore invalidate the earlier comparison.

Solution 8a: Search architectures and FL settings together. An alternating procedure can choose a training setup, search an architecture under it and use the result to choose the next setup [feathers_2023]. Evolutionary and black-box strategies can instead search the architecture together with settings such as local epochs and client participation, then evaluate each combination through federated training [aging_fednas_2025, breastcancer_fednas_2024]. Candidate scores then reflect the FL procedure in which the architecture will operate.

Candidate scores can also change with the current search state. Layer-wise updates

may favor different operations at different depths, and an early preference for operations without parameters can prevent convergence [remote_sensing_fednas_2025]. Fixing one operation changes the computation graph and weight sharing, so the next choice may be evaluated before the remaining weights adapt [spider_2021]. Changing which subnets are trained can likewise make aggregated updates unstable and slow convergence [divide-and-conquer_fednas_2023]. A score can therefore reflect when and how a candidate was evaluated, not only its architecture.

Solution 8b: Regularize layer-wise architecture updates. During local differentiable search, an added term can align layer gradients with architecture parameters. This reduces depth-dependent preferences for particular operations before client updates are aggregated [remote_sensing_fednas_2025].

Solution 8c: Insert recovery rounds between architecture decisions. A progressive supernet search can first warm up the shared weights and then fix one operation at a time based on local validation accuracy. After each decision, several recovery rounds update the remaining weights before the next edge is evaluated [spider_2021]. Unlike Solution 2b, this pause helps weights adapt to each architecture change. Once all relevant edges are resolved, ordinary training continues with a fixed architecture.

Solution 8d: Vary which subnets are trained over time. The server can assign very different subnets in early rounds and increasingly similar ones as training advances. Early rounds explore separate parts of the search space, while later rounds concentrate on better understood regions [divide-and-conquer_fednas_2023]. These structured assignments also combine more consistently than independently sampled subnets.

5.4 Participation Dynamics

The participating clients and the timing of their updates can change throughout architecture search. Clients may disconnect, respond at different speeds, converge at different times or participate with different frequencies.

Challenge 9: Variable Client Participation Disrupts Search

Clients can become unavailable because of changing network conditions, battery limits or competing local work. A search that requires the same clients in every round may stall or lose updates for candidates or supernet paths assigned to disconnected clients [optimize_fednas_edge_2021].

Available clients may still complete search work at different speeds. A round may wait for the slowest client, while an update that arrives after the global search has advanced may already be outdated [nasfly_2024, rl_fednas_2021, fedregnas_2025].

This differs from resource infeasibility, where a client cannot run the workload, and dropout, where it returns no update.

Solution 9a: Group split-model segments by training time. Split-model strategies can measure client computation and communication times and group segments that finish at similar times. Unlike Solution 6b, this groups clients only by observed timing so that supernet training waits less for the slowest segment [nasfly_2024].

Solution 9b: Adjust the influence of outdated search updates. The server can account for delay when adding a late update after the global search has advanced [rl_fednas_2021]. Differentiable search can instead give older updates less weight [fedregnas_2025]. Unlike Solution 6a, these approaches adjust an update according to its age rather than how much each parameter was trained.

In personalized differentiable search, clients can learn architecture parameters at different rates because their data and local training differ. One shared stopping round can end some searches too early while allowing others to overfit, for example by increasingly favoring skip connections [collaborative_nas_for_pfl_2025]. Clients can therefore need different stopping times even when their updates arrive promptly.

Solution 9c: Stop architecture updates independently. After a shared warm-up period, each client can stop updating its architecture when a local test detects growing preference for skip connections. Other clients continue until they meet their own stopping condition [collaborative_nas_for_pfl_2025].

Partial participation and random client selection can cause some clients to contribute more often than others [optimize_fednas_edge_2021, peaches_2024]. Even if the search continues, frequently available clients influence shared weights and architecture rankings more often. The result may therefore poorly represent clients that participate less often.

Cross-cutting solution 6b. This solution can select participants by both resources and data coverage so that availability alone does not determine whose data enter the search [network_aware_fed_nas_2025, finch_2024].

5.5 Search Security

NAS-in-FL communicates information about the architecture in addition to conventional model updates. Malicious participants can interfere with the search, while updates from honest clients may reveal private information.

Challenge 10: Architecture Search Introduces New Security Risks

NAS-in-FL creates additional attack targets because clients influence both model weights and search signals that determine the selected architecture. Malicious clients

can manipulate either through poisoned data or local training, steering the search toward a model that supports a backdoor attack. Weight sharing can spread poisoned weights across candidates. Fourfold-scaled updates from one of eight clients degraded two discrete strategies, showing that poisoning can persist in the selected architecture [privacy_preserving_fednas_for_edge_2025].

Conditional observation. During differentiable supernet-based search, each candidate operation remains in a weighted mixture while clients update its weights and selection probability. [privacy_preserving_fednas_for_edge_2025] finds that an operation contributes less when its poisoned weights work against or contribute little to the main task.

[privacy_preserving_fednas_for_edge_2025] demonstrates this behavior only for the studied backdoor attacks and search spaces. Retraining the selected fixed architecture from scratch did not preserve its resistance to attack, suggesting that the effect belongs to the trained search state rather than the architecture itself.

Solution 10a: Pre-screen search-space modules on server data. The server can use its own data to select a compact set of accurate, efficient modules before federated search. In [privacy_preserving_fednas_for_edge_2025], this reduced complexity and limited overfitting to noise from backdoor triggers. Unlike Solution 1a, the mechanism tests modules using data. Unlike Solution 4b, it removes modules before search instead of changing their scores.

NAS-in-FL transmits data-dependent architecture gradients and rewards that fixed-architecture FL does not [dp-fnas_2020, feathers_2023]. Other strategies transmit structural encodings or personalized choices that may reveal characteristics of client data, individual examples or which client sent an update, although [multi-granular_fednas_2023, privacy_preserving_fednas_for_edge_2025] does not demonstrate such attacks. The risk depends on what is transmitted and the attacker’s capabilities.

Solution 10b: Apply differential privacy to search signals. Differentiable strategies can limit the size of model and architecture gradients and add Gaussian noise before upload. Noise can also be added to rewards used for hyperparameter selection [dp-fnas_2020, feathers_2023]. In vertically partitioned search, the intermediate activations and backward gradients exchanged between parties can receive the same protection [self-supervised_vertical_fednas_2023]. The noise can, however, reduce model and search performance [privacy_preserving_fednas_for_edge_2025].

Solution 10c: Apply secure aggregation to matching search variables. When every client updates the same differentiable supernet, model weights and architecture parameters have matching structures and can use the same secure-aggregation protocol [privacy_preserving_fednas_for_edge_2025]. This hides each client’s updates from the server but requires every client to use the same search representation.

Cross-cutting solution 7b. The compact structural encoding exchanged in this solu-

tion may make it harder to reconstruct inputs because it has far fewer dimensions than the private data. The cited analysis argues that a small derivative matrix can correspond to many image batches, making exact pixel recovery by that procedure impractical [multi-granular_fednas_2023]. However, a compact encoding provides no formal privacy guarantee. It may still reveal properties of the client data or the source of an update and may require differential privacy or encryption. Table 5.1 summarizes the challenges and solutions. Cross-cutting entries indicate where a solution primarily discussed under another challenge also applies.

Table 5.1: Overview of the identified challenges, their primary solutions, and cross-cutting applications.

Theme	Challenge	Solution	Example publication
Client resource constraints	Challenge 1	Solution 1a	[fed_auto_mri_2023]
		Solution 1b	[fedoras_2022]
		Solution 1c	[fednaspet_2023]
		Cross-cutting solution 7d	[hapfnas_2025]
	Challenge 2	Solution 2a	[decnas_2022]
		Solution 2b	[dfnas_2020]
		Solution 2c	[hapfnas_2025]
		Solution 2d	[decnas_2022]
		Solution 2e	[medical_image_segmentation_fednas_2024]
		Solution 2f	[nasfly_2024]
		Solution 2g	[dfnas_2020]
	Challenge 3	Solution 3a	[rt_fed_evo_nas_2020]
		Solution 3b	[fedregnas_2025]
		Solution 3c	[finch_2024]
		Cross-cutting solution 4b	[multi_objective_evo_fl_2020]
Client heterogeneity	Challenge 4	Solution 4a	[superfednas_2024]
		Solution 4b	[clustered_direct_fednas_2022]
	Challenge 5	Solution 5a	[electricity_load_forecasting_fl_darts_2023]
		Solution 5b	[superfednas_2024]
		Solution 5c	[fedpnas_2021]
		Cross-cutting solution 6b	[finch_2024]
	Challenge 6	Solution 6a	[fedoras_2022]
		Solution 6b	[finch_2024]
		Cross-cutting solution 7d	[hapfnas_2025]
Decentralized search state	Challenge 7	Solution 7a	[fednas_2021]
		Solution 7b	[multi-granular_fednas_2023]
		Solution 7c	[f2nas_2024]
		Solution 7d	[f2nas_2024]

5 Challenges and Solutions for Using NAS in FL

Table 5.1 continued

Theme	Challenge	Solution	Example publication
	Challenge 8	Solution 8a	[feathers_2023]
		Solution 8b	[remote_sensing_fednas_2025]
		Solution 8c	[spider_2021]
		Solution 8d	[divide-and-conquer_fednas_2023]
<i>Participation dynamics</i>	Challenge 9	Solution 9a	[nasfly_2024]
		Solution 9b	[rl_fednas_2021, fedregnas_2025]
		Solution 9c	[collaborative_nas_for_pfl_2025]
		Cross-cutting solution 6b	[network_aware_fed_nas_2025]
<i>Search security</i>	Challenge 10	Solution 10a	[privacy_preserving_fednas_for_edge_2025]
		Solution 10b	[dp-fnas_2020]
		Solution 10c	[privacy_preserving_fednas_for_edge_2025]
		Cross-cutting solution 7b	[multi-granular_fednas_2023]

6 Reusable Solution Mechanisms

A NAS-in-FL scenario is a tuple of a NAS method type and FL system characteristics. The NAS method type comprises its search strategy, search space and performance-estimation strategy. The FL system characteristics follow the dimensions defined in the background. Each row in Table 6.1 describes the broadest scenario range in which the solution mechanism can operate. “Any” includes every value defined for that characteristic. A parenthetical condition narrows this range only where the mechanism requires an additional property. These entries are conceptual compatibility judgments and do not claim that every included combination has been implemented or evaluated.

Table 6.1: Broadest possible NAS-in-FL scenario for which each solution is applicable.

Solution	Search strategy type	Search space type	Performance estimation strategy	Data heterogeneity	Data balance	Client computation resources	Network environment	Federation environment	Deployment environment	Trust and privacy requirements
S1a – Use resource-efficient search components	Any	Any (permits restriction to inexpensive components)	Any	Any	Any	Homogeneous or heterogeneous, with binding client limits	Any	Any	Any	Any
S1b – Send clients subnets tailored to their capacity	Supernet-Based Methods	Any (supernet-encodable and supports capacity-matched subnets)	One-Shot Models	Any	Any	Heterogeneous, with at least one constrained client tier	Any	Any	Any	Any
S1c – Search trainable adaptation modules over a frozen backbone	Any	Any (modular search over a transferable frozen backbone)	Any	Any (related target tasks)	Any	Homogeneous or heterogeneous, with binding client limits	Any	Any	Any	Any
S2a – Allocate candidate evaluation budgets adaptively	Black-Box Optimization Techniques	Any	Multi-Fidelity	Any	Any	Homogeneous or heterogeneous, with a constrained candidate-evaluation budget	Any	Any	Any	Any
S2b – Prune the supernet progressively	Supernet-Based Methods	Any (supernet-encodable and progressively prunable)	Any (must rank operations or subnets)	Any	Any	Homogeneous or heterogeneous, with a constrained search budget	Any	Any	Any	Any

Continued on next page

Table 6.1 continued

Solution	Search strategy type	Search space type	Performance estimation strategy	Data heterogeneity	Data balance	Client computation resources	Network environment	Federation environment	Deployment environment	Trust and privacy requirements
S2c – Predict candidate performance	Any	Any (predictor-encodable)	Surrogate Models	Any (client-specific or collaborative prediction may be required under heterogeneity)	Any	Homogeneous or heterogeneous, with a constrained candidate-evaluation budget	Any	Any	Any	Any
S2d – Evaluate independent candidates or subnets in parallel	Black-Box Optimization Techniques and Non-Differentiable Supernet-Based Methods	Any	Any (independent candidate estimates)	Any	Any	Homogeneous or heterogeneous, with sufficient aggregate client capacity	Any	Stable or dynamic, with sufficient simultaneous availability	Any	Any
S2e – Reuse search-trained weights for candidate evaluation and deployment	Supernet-Based Methods	Any (supernet-encodable)	One-Shot Models and Weight Inheritance	Any	Any	Any	Any	Any	Any	Any
S2f – Use split-training idle periods for local search	Any	Any (decomposable into locally searchable split-model segments)	Any	Any	Any	Homogeneous or heterogeneous, with sufficient capacity for local segment search	Any (split execution creates sequential idle periods)	Any	Any	Any

Continued on next page

Table 6.1 continued

Solution	Search strategy type	Search space type	Performance estimation strategy	Data heterogeneity	Data balance	Client computation resources	Network environment	Federation environment	Deployment environment	Trust and privacy requirements
S2g – Jointly derive an architecture and its deployable weights	Supernet-Based Methods	Any (supernet-encodable)	One-Shot Models	Any	Any	Any	Any	Any	Any	Any
S3a – Identify known candidates with compact selectors or genotypes	Any (supports discrete candidate instantiation)	Any (compactly encodable and clients retain a compatible model)	One-Shot Models and Weight Inheritance	Any	Any	Homogeneous or heterogeneous, with capacity to retain the compatible model	Homogeneous or heterogeneous, with binding transfer limits	Any	Any	Any
S3b – Synchronize architecture parameters less frequently	Differentiable Supernet-Based Methods	Any (supernet-encodable)	One-Shot Models	Any	Any	Any	Homogeneous or heterogeneous, with binding transfer limits	Stable or dynamic, with repeated synchronized rounds	Any	Any
S3c – Aggregate search updates hierarchically	Any (produces aggregatable distributed feedback)	Any	Any	Any	Any	Any	Homogeneous or heterogeneous, with a hierarchical topology and constrained wide-area links	Stable or dynamic, with an intermediate aggregation tier	Any	Any
S4a – Train one elastic supernet for constraint-specific deployment	Supernet-Based Methods	Any (elastic supernet-encodable)	One-Shot Models	Any	Any	Heterogeneous	Any	Any	Heterogeneous	Any

Continued on next page

Table 6.1 continued

Solution	Search strategy type	Search space type	Performance estimation strategy	Data heterogeneity	Data balance	Client computation resources	Network environment	Federation environment	Deployment environment	Trust and privacy requirements
S4b – Include deployment-resource costs in the architecture objective	Any (supports a resource-aware objective)	Any	Any (resource cost is observable or predictable)	Any	Any	Any	Any	Any	Uniform or heterogeneous, with an explicit deployment-resource budget	Any
S5a – Cluster clients by learned architecture preference	Differentiable Supernet-Based Methods	Any (shared architecture-parameter space)	One-Shot Models	Heterogeneous	Any	Any	Any	Any	Any	Any
S5b – Optimize the worst-performing subnets across client partitions	Any (supports a worst-partition objective)	Any	Any (partition-specific performance evidence)	Heterogeneous	Any	Any	Any	Any	Any	Any
S5c – Factor the architecture into shared and task-specific components	Any	Any (factorable into shared and task-specific components)	Any	Any (clients solve related tasks requiring task-specific components)	Any	Any	Any	Any	Any	Any
S6a – Aggregate parameters according to their actual exposure	Supernet-Based Methods	Any (aligned shared weights with partial operation exposure)	One-Shot Models	Any	Any	Heterogeneous, with capacity-dependent operation or subnet eligibility	Any	Any	Any	Any

Continued on next page

Table 6.1 continued

Solution	Search strategy type	Search space type	Performance estimation strategy	Data heterogeneity	Data balance	Client computation resources	Network environment	Federation environment	Deployment environment	Trust and privacy requirements
S6b – Form resource-feasible, data-representative client groups	Any	Any	Any (client-group-based performance evidence)	Heterogeneous	Any	Homogeneous or heterogeneous, with resource-feasible grouping	Homogeneous or heterogeneous, with latency or bandwidth constraints included in grouping	Stable or dynamic, with enough available clients to form representative groups	Any	Any
S7a – Aggregate aligned architecture and model variables together	Supernet-Based Methods (shared architecture state)	Any (aligned shared supernet)	One-Shot Models	Any	Any	Any	Any	Stable or dynamic, with an aligned participating search state	Any	Any
S7b – Exchange and aggregate discrete architecture encodings separately from model parameters	Black-Box Optimization Techniques and Non-Differentiable Supernet-Based Methods	Any (discretely encodable)	Any	Any (clients produce distinct architecture choices)	Any	Any	Any	Any	Any	Any
S7c – Aggregate aligned blocks by parameter-distribution similarity	Any (supports shared or inherited compatible weights)	Any (contains structurally corresponding blocks)	One-Shot Models and Weight Inheritance	Any	Any	Any	Any	Any	Any (local architectures may differ)	Any

Continued on next page

Table 6.1 continued

Solution	Search strategy type	Search space type	Performance estimation strategy	Data heterogeneity	Data balance	Client computation resources	Network environment	Federation environment	Deployment environment	Trust and privacy requirements
S7d – Transfer knowledge across architecture boundaries through distillation	Any	Any	Any	Any	Any	Any	Any	Any	Any	Data locality only or additional confidentiality or integrity requirements, with permissible common distillation inputs
S8a – Search architectures and FL control variables jointly	Any (architecture and FL variables are jointly optimizable)	Any (extended with FL control variables)	Any	Any	Any	Any	Any	Stable or dynamic, with the relevant participation variables included in the search	Any	Any
S8b – Regularize layer-wise architecture updates	Differentiable Supernet-Based Methods	Any (shared supernet)	One-Shot Models	Heterogeneous	Any	Any	Any	Any	Uniform (one shared architecture)	Any
S8c – Separate structural decisions with recovery rounds	Supernet-Based Methods and Weight-Inheriting Black-Box Optimization	Any	One-Shot Models and Weight Inheritance	Any	Any	Homogeneous or heterogeneous, with sufficient capacity for recovery training	Any	Stable across recovery rounds	Any	Any

Continued on next page

Table 6.1 continued

Solution	Search strategy type	Search space type	Performance estimation strategy	Data heterogeneity	Data balance	Client computation resources	Network environment	Federation environment	Deployment environment	Trust and privacy requirements
S8d – Vary subnet coverage over the search	Supernet-Based Methods	Any (mask-or path-encodable supernet)	One-Shot Models	Any	Any	Heterogeneous	Any	Stable or dynamic, with repeated client sampling	Any	Any
S9a – Group split-model segments by training latency	Any	Any (decomposable into split-model segments)	Any	Any	Any	Heterogeneous	Heterogeneous	Any	Any	Any
S9b – Compensate or down-weight stale search updates	Any (iterative search accepts late feedback)	Any	Any	Any	Any	Any	Heterogeneous latency	Stable or dynamic, with late updates	Any	Any
S9c – Stop architecture updates independently	Any (supports client-specific stopping and selection)	Any	Any	Heterogeneous	Any	Any	Any	Stable or dynamic	Any	Any
S10a – Pre-screen search-space modules on server data	Any	Any (contains server-screenable modules or operations)	Any	Any	Any	Any	Any	Any	Any	Additional integrity requirements and representative server validation data
S10b – Apply differential privacy to search signals	Any (communicated search signals are protectable)	Any	Any	Any	Any	Any	Any	Any	Any	Additional confidentiality requirements

Continued on next page

Table 6.1 continued

Solution	Search strategy type	Search space type	Performance estimation strategy	Data heterogeneity	Data balance	Client computation resources	Network environment	Federation environment	Deployment environment	Trust and privacy requirements
S10c – Securely aggregate aligned search variables	Any (uses aligned aggregatable numerical search state)	Any (aligned search variables)	Any	Any	Any	Any	Any	Stable or dynamic, with a secure-aggregation-compatible cohort	Uniform (one shared architecture state)	Additional confidentiality requirements

7 Discussion

- the further the FL system deviates from centralised NAS, the more challenging it is => conversely, when NAS-in-FL degenerates to centralised NAS in some aspects it becomes simpler - show how some solutions limit compatibility with others and how they are - the search space trade-off - suprisingly many NAS-in-FL methods do X - we expected federation scale to play a more prominent role, but this was barely ever considered by NAS-in-FL methods and the largest federation scale currently in the literature is ..., significantly smaller than regular FL setups - similar to NAS research, supernet-based methods have become more popular over time - performance evaluation is basically universally just training a candidate architecture for a couple of epochs, making the main differentiating factors for challenge applicability the NAS search strategy type and secondarily the search space - The FL system characteristics which NAS-in-FL methods apply to tend to be underspecified

7.1 Principal Findings

7.2 Implications for Practice and Research

- clear guidance on existing related solutions and challenges and the context/circumstances under which challenges arise - potentil for re-usable solutions to challenges

7.3 Limitations

- rely on literature for facts, i.e. described only challenges and solutions from the literature - few NAS-in-FL methods establish an explicit causal link between FL system characteristics and challenges, therefore this needs to be treated as corrolation evidence, not causal

7.4 Future Research

- large share

8 Conclusion

- the amount of possible knobs to turn in NAS-in-FL is astronomical and the literature has only investigated a small subset of them - NAS in FL trails behind NAS and FL - provide some clear direction for future research

Abbreviations

List of Figures

List of Tables